PETWELL

Frontend Addendum

Changes since Revision 1

New routes, components and data flow added to the Next.js app after the first edition of the frontend documentation. Read alongside PetWell_Frontend_Documentation.pdf, which remains accurate for everything it covers.

Next.js 16 · React 19 · next-auth v5 · Tailwind 4

Addendum · July 2026
Pet Health Passport & Appointment Management System

Contents

1 Scope of this addendum 3

2 New routes 3

3 New components 3

4 Patient lookup 4

5 Appointment detail and history 4

6 Slot capacity in booking 4

Flow . . . 4

Reading the slot data . . . 5

Staleness and the 409 . . . 5

A note on the effect . . . 5

7 Updated inventories 6

Routes . . . 6

API and action surface added . . . 6

1 Scope of this addendum

`PetWell_Frontend_Documentation.pdf` was written on 29 July 2026. Its account of the architecture — the layering from Server Component through API client to Server Action, the `next-auth` session handling, `middleware.ts` route protection, and the styling approach — is still correct and is not repeated here.

Three pieces of work landed afterwards, and this addendum documents them:

- **Patient lookup** — a search screen so clinic staff can find any pet directly.
- **Appointment detail and status history** — a route per appointment, rendering the audit trail.
- **Slot capacity in booking** — the free-form time input replaced by an availability-aware hour grid.

The route and component inventories in Revision 1 should be read as superseded by the tables below.

2 New routes

Route	Area	Renders
`/staff/pets`	Staff	Patient lookup — search by pet name, microchip, or owner name / email / phone
`/owner/appointments/[appointmentId]`	Owner	Appointment detail plus status history
`/staff/appointments/[appointmentId]`	Staff	The same detail, with staff actions

Both appointment detail routes are dynamic segments alongside a static sibling. `/owner/appointments/book` still resolves to `book/page.tsx` rather than being captured as an `appointmentId`, because Next.js gives static segments priority over dynamic ones.

The staff sidebar gained a **Patients** entry between Home and Clinic Profile. `AppShell` marks a nav item active with `pathname.startsWith(href)`, so `/staff/pets/[petId]` correctly highlights Patients.

3 New components

Component	Kind	Purpose
`features/appointment/SlotPicker`	Client	Hour grid for a clinic day; disables full and past slots
`features/appointment/AppointmentDetail`	Server	Summary card plus audit trail; shared by both areas

Component	Kind	Purpose
`features/appointment/StatusTimeline`	Server	Renders `AppointmentStatusLog[]` as a dotted rail

`AppointmentDetail` takes an `area` prop of `'owner'` or `'staff'`, which decides the back link and which actions render — `CancelButton` for owners, `StatusActions` for staff. One component serves both routes rather than duplicating the layout.

`StatusTimeline` colours each dot by status, reusing the palette from `components/shared/StatusBadge.tsx` so a status looks the same wherever it appears. It also renders the appointment's `createdAt` as an implicit first "Requested by owner" event, since that moment predates any log row.

4 Patient lookup

`/staff/pets` is a Server Component reading `searchParams.q`, with a plain `method="get"` form — no client state, so a search is shareable and survives a reload. It calls `PetApi.search(q)`, which hits `GET /pets/search?q=`.

Results show the owner's name, phone and the pet's microchip number, because staff need a detail to confirm they have the right patient. An allergy badge appears when `drugAllergy` is set. An empty `q` renders guidance rather than firing a request.

5 Appointment detail and history

```
page.tsx  →  AppointmentApi.get(id)      →  GET /appointments/:id
          →  AppointmentApi.logs(id)     →  GET /appointments/:id/logs
          →  <AppointmentDetail appointment logs area />
```

Both calls are wrapped in `.catch()`: a missing appointment triggers `notFound()`, while a failed log fetch degrades to an empty timeline rather than taking the page down.

The status-log endpoint now returns a `changedBy` summary — id, first name, last name, role — so the timeline can name who moved the appointment instead of only when.

Because the detail pages are cached like any other route, `cancelAppointmentAction` and `updateAppointmentStatusAction` now call `revalidatePath` for the detail path as well as the list, otherwise a status change would leave a stale timeline behind.

6 Slot capacity in booking

Each clinic accepts two appointments per one-hour slot, enforced by the API. The booking form was reworked so a user is not led into a rejection.

Flow

The form now collects pet, clinic, **date**, then **hour** — where before it collected a single free-form datetime.

```
BookAppointmentForm
  ├─ DatePicker  mode="date"   disabledDays={{ before: today }}
  └─ SlotPicker  clinicId + date
       └─ getSlotAvailabilityAction(clinicId, date)     'use server'
            └─ AppointmentApi.availability()  →  GET /clinics/:id/availability?date=
```

`getSlotAvailabilityAction` returns `DayAvailability | null`, returning null on an `ApiError` so the picker can render "couldn't load" instead of throwing inside an effect.

Reading the slot data

Each slot carries `slotStart` (an absolute ISO instant), `hour` (the clinic-local hour), `booked`, `remaining`, `isFull` and `isPast`.

> **The client does no timezone arithmetic**
> Labels are rendered from the `hour` number, not by formatting `slotStart`. Formatting the instant would render it in the visitor's own timezone, so a user whose laptop is set to another region would see the wrong clinic time. Selecting a slot posts its `slotStart` back verbatim as `appointmentDate`.

Staleness and the 409

Availability is a snapshot. Between loading the grid and pressing submit, another owner may take the last seat, and the API answers 409 with "This time slot is fully booked. Please choose another time."

That path was already handled by the existing error plumbing — `apiFetch` turns any non-2xx into an `ApiError`, the Server Action returns it as an `ActionResult`, and the form renders the message. Two additions complete it: the selection is cleared, and a `refreshKey` is bumped so `SlotPicker` refetches and the hour visibly flips to **Full**.

`bookAppointmentAction` also now surfaces the first Zod issue rather than a generic "please complete the booking form", so a past date or an invalid slot explains itself.

A note on the effect

`SlotPicker` tags each fetch result with the request that produced it:

```
const requestKey = `${clinicId}|${date}|${refreshKey}`;
// ...
const fresh = loaded?.key === requestKey ? loaded : null;
```

Changing clinic or date changes `requestKey`, which invalidates the previous result by derivation. An earlier version cleared state inside the effect instead, which the React Compiler lint correctly rejects — synchronous `setState` in an effect body causes cascading renders. Deriving staleness removed that reset effect and a second one that had been clearing an invalid selection.

7 Updated inventories

Routes

Route	Area	Purpose
`/`	Public	Landing
`/login`	Public	Sign in
`/register`	Public	Sign up, owner or clinic staff
`/owner/dashboard`	Owner	Pets and upcoming appointments
`/owner/pets`	Owner	Pet list
`/owner/pets/new`	Owner	Add pet
`/owner/pets/[petId]`	Owner	Pet detail and health passport
`/owner/pets/[petId]/edit`	Owner	Edit pet
`/owner/clinics`	Owner	Clinic search and filter
`/owner/clinics/[clinicId]`	Owner	Clinic detail
`/owner/appointments`	Owner	Appointment list
`/owner/appointments/book`	Owner	Booking form with slot picker
`/owner/appointments/[appointmentId]`	Owner	Detail and status history
`/owner/profile`	Owner	Profile
`/staff/dashboard`	Staff	Pending and today counters
`/staff/pets`	Staff	Patient lookup
`/staff/pets/[petId]`	Staff	Patient record and add-record form
`/staff/clinic`	Staff	Clinic profile
`/staff/appointments`	Staff	Appointment queue
`/staff/appointments/[appointmentId]`	Staff	Detail and status history
`/staff/profile`	Staff	Profile

API and action surface added

Layer	Addition
`lib/api/pet.api.ts`	`search(q)`
`lib/api/appointment.api.ts`	`availability(clinicId, date)`
`lib/actions/appointment.action.ts`	`getSlotAvailabilityAction`

Layer	Addition
`lib/api/api.type.ts`	`PetSearchOwner`, `PetSearchResult`, `SlotAvailability`, `DayAvailability`, `StatusLogAuthor`
`components/ui/date-picker.tsx`	`disabledDays` and `startMonth` props

`DatePicker` gained those two props rather than hardcoding a future-only constraint, because the same component also picks a pet's birth date, which must be in the past.